WriteOnce Blog Engine

Summary

The WriteOnce Blog Engine is a flexible and secure blog engine, similar to WordPress, but with a much more powerful way of handling how posts are written, stored and displayed. The innovative part of the project is that the blog posts are stored as 'intermediate representation (IR)' that allows them to be written in a range of languages, but also displayed and exported in other formats in the future, whilst still maintaining their formatting. In this 12 week project we will develop a prototype of the Blog Engine and if time allows add functionality around accessibility, security and improvements to the user interface.

Team roles

Project Manager: Paula Blackledge

Technical Lead: Mei Happs

Frontend Developer: Max De La Nougerede

Backend Developer: Tim Bartlett

Intended audience

The blog engine is an ideal solution for writers and organisations that need flexible ways to write blogs. They can write the blogs in a format that suits them whilst still producing clean and well formatted posts, that meet good standards by default. For those working on a blog within a team, the flexibility of the IR means different people can work on the blogs in their preferred format and it also future proofs the blogs as the post can be re-exported in multiple formats.

Constraints of the approach

Our WriteOnce Blog Engine will be based around a single structured representation, so a major constraint is that we may not be able to represent all required structural text features within this model. Alongside this, because of time restrictions, it is unlikely we will produce a version on WriteOnce that will provide a rich interface for blog editors. These constraints are traded off by the long-term advantages of the approach that allows for easy conversion to other formats whilst still maintaining its structure.

Core features

High priority features:

- IR implementation, priority IR to HTML
- Storage (SQL)
- Authentication/user accounts
- Basic editor function
- Markdown export
- HTML import

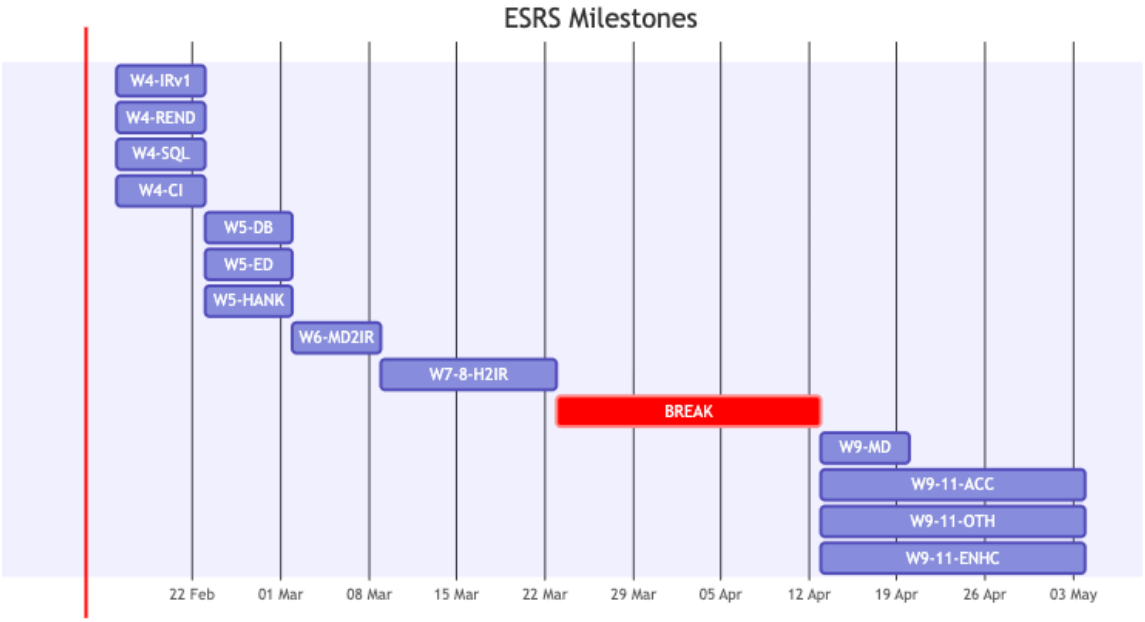
Lower priority features:

- Accessibility checker
- Import/export other formats
- Improvements to blog editor

- Custom parser

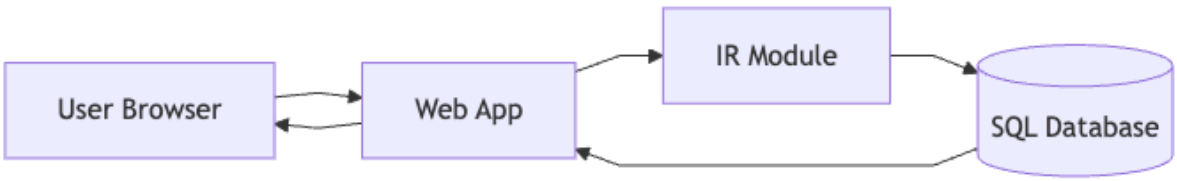
Milestones

- Decide on systems language for IR handling (by 13th February) - LANG
- V1 of specification document for hand in (by 13th February) - SPEC
- V1 schema for IR (week 4) - IRv1
- IR to HTML renderer (week 4) - REND
- Define SQL schema (week 4) - SQL
- Setup CI (week 4) - CI
- Initial SQL DB setup and integration (week 5) - DB
- Basic editor (week 5) - ED
- Authentication via Hanko (week 5) - HANK
- Markdown to IR supported (week 6) - MD2IR
- HTML to IR supported (week 7+8) - H2IR
- IR to Markdown supported (week 9) - MD
- Accessibility checker (week 9-11) - ACC
- IR to other formats (week 9-11) - OTH
- Editor enhancements (week 9-11) - ENHC



Infrastructure

Architecture Diagram



Routing

We'll need some form of routing. Exact router is still to be decided on.

The routes we would need to provide would be:

- a home page (this would probably list blog posts in some searchable and/or ordered manner)
- a general route for blog posts
- a route for the secure admin area

Post formatting and storage

This will partially come down to how we decide to format the posts. We have many choices here, such as:

- Plaintext with predefined plaintext sequences for “special” datatypes. i.e. in Markdown i think you can use something like:

  ```
  ![[image_name]]
  ```

  to get images inserted into your plaintext.
- Some structured format like json or some other languages object notation in which a similar kind of thing would be encoded with:

  ```
  [
    {
      "type": "plaintext",
      "content": "some text"
    },
    {
      "type": "image",
      "image_path": "./imgs/some_image.png"
    },
    {
      "type": "plaintext",
      "content": "some more text"
    }
  ]
  ```
- raw HTML (not preferred due to maintainability and security concerns)

The posts will be stored in a SQL database.

Security

Security for users is a priority for this system, but due to the short timeline and the need to prioritise the core element of the project (The IR development), we propose to use Hanko as a pre-built option (https://github.com/teamhanko/hanko), as this provides with an easy to use but very secure and easy to use solution that is built on privacy first principles. Alternatives we considered were: Sessions and JWT.

Scalability

Although WriteOnce is a prototype system, we have designed it with scalability and long-term maintainability in mind. Scalability considerations apply primarily to read performance, write reliability, and architectural extensibility.

Read Scalability

Public blog viewing is expected to generate significantly more traffic than administrative writes.

To ensure performance as the number of posts increases:

- Post listings will be paginated to avoid large result sets.

- Frequently queried fields such as `slug` and `published_at` will be indexed in the SQL database.
- Where appropriate, rendered HTML output from the IR may be cached to avoid repeated IR-to-HTML transformations for frequently accessed posts.

This ensures that growth in content volume does not degrade user-facing performance.

Write Scalability
Administrative actions (create, edit, delete) occur less frequently but must prioritise reliability.

- Write operations will be executed transactionally within the SQL database.
- The IR will be validated before persistence to ensure structural integrity.
- Authentication is delegated to Hanko, reducing custom security overhead and improving maintainability.

Architectural Scalability
The backend is designed to be largely stateless:

- Persistent state is stored exclusively in the SQL database.
- This separation allows multiple application instances to be deployed behind a load balancer if needed.
- Offloading authentication to Hanko further reduces server-side state management complexity.

This design allows horizontal scaling without significant architectural redesign.

IR Extensibility
The intermediate representation is implemented as a discriminated union representing an abstract syntax tree (AST).

- New node types can be added without breaking existing documents.
- The schema may be versioned to support future evolution.
- Unsupported constructs encountered during import will be handled via controlled fallback mechanisms.

This ensures long-term extensibility and structural consistency.

Error Handling
The WriteOnce system will distinguish between expected operational failures and unexpected exceptional conditions, applying different error handling strategies to each category.

Result Types for Expected Failures
Domain logic operations that may fail as part of normal operation return Result types, forcing callers to explicitly handle both success and failure cases at compile time:

- IR validation: Structural validation of intermediate representation nodes
- Format conversion: Parsing and transformation between IR and external formats (Markdown, HTML)
- Import operations: User-provided content that may be malformed or unsupported
- Export operations: IR constructs that cannot be represented in the target format

Example Result type definition:

```typescript
type Result<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };

function parseMarkdownToIR(markdown: string): Result<IRNode[], ParseError> {
  // Returns Result forcing caller to handle parse failures
}
```

This approach leverages TypeScript's discriminated unions and `ts-pattern` for exhaustive pattern matching, ensuring error cases cannot be accidentally ignored. We will likely use `ts-results` for our Result types.

Exceptions for Exceptional Conditions

Exceptions are reserved for genuinely unexpected failures that indicate system-level problems or programming errors:

- Database connection failures
- File system errors during server startup
- Authentication service (Hanko) unavailability
- Malformed data from trusted internal sources

These failures typically cannot be meaningfully recovered from at the call site and are handled by centralized error handlers at architectural boundaries (HTTP middleware, database connection layer).

Conversion at Boundaries

HTTP route handlers convert between these approaches:

- Result type failures from domain logic are converted to appropriate HTTP error responses (400, 422)
- Exceptions are caught by middleware and converted to 500 responses with sanitized error messages

This strategy provides compile-time safety for business logic while maintaining idiomatic exception handling at the framework level.

Intermediate Representation

This will be the core of the project.

With an intermediate representation, we can allow for many different "views" of a post, i.e. the intermediate representation isn't readable or directly usable but can easily be converted to and from HTML (priority no. 1), Markdown, org and other plaintext formats. While viewing the post in another format, it's important to remember that the view is not a source of truth, just another way of looking at the intermediate representation for readability.

Here is an example of how a markdown document could be losslessly represented in our format:

```
# Example header

Some example paragraph.

- A
* Bulleted
- List
```

```
  - Maybe with
  - Indentation levels

1. Or even a
2. Numbered one
  - Maybe even
  - With a bulleted list under it
```

```
Header(level: 1, "Example header"),
LineBreak(),
"Some example paragraph.",
LineBreak(),
BulletedList([
  Bullet(bullet_character: {BulletCharacter::Markdown::Hyphen}, "A"),
  Bullet(bullet_character: {BulletCharacter::Markdown::Asterisk}, "Bulleted"),
  Bullet(bullet_character: {BulletCharacter::Markdown::Hyphen}, "List"),
  BulletedList(
    Bullet(bullet_character: {BulletCharacter::Markdown::Hyphen}, "Maybe with"),
    Bullet(bullet_character: {BulletCharacter::Markdown::Hyphen}, "Indentation
levels")
  )
]),
LineBreak(),
NumberedList([
  Number("Or even a"),
  Number("Numbered one"),
  BulletedList([
    Bullet(bullet_character: {BulletCharacter::Markdown::Hyphen}, "Maybe even"),
    Bullet(bullet_character: {BulletCharacter::Markdown::Hyphen}, "With a bulleted
list under it")
  ])
])
```

```
Header(level: int, str)
LineBreak()
BulletedList(list[Bullet | BulletedList | NumberedList])
Bullet(bullet_character: set[BulletCharacter], str)
NumberedList(list[Number | NumberedList | BulletedList])
Number(str)

BulletCharacter {
  Markdown {
    Hyphen => "-", # default
    Asterisk => "*"
  }
}
```

This example is not a perfect representation, but it should hopefully get the idea across. The types need to be exhaustive enough to cover everything in each domain while also ensuring it's possible to map every domain to the intermediate representation.

As an example, bullet_character is a set of enforced characters. This is because one domain may not have more than one bullet character, and therefore wouldn't enforce a bullet character when you write in it but other domains, like Markdown, do. This approach allows for you to write in one format that has multiple bullet characters, edit in another format that has multiple bullet characters and have each time you view it maintain the

characters you chose in both languages while also allowing languages with only one bullet character to not have to enforce any kind of memory.

In terms of actually parsing these formats, we considered using tree-sitter as it already has predefined grammars for most other formats, however tree-sitter is by design overly permissive, which is good for syntax highlighting but not for conversion between formats. In the end we have decided to use a traditional parser, likely ANTLR as we have experience in it from our compilers module and it can be used within typescript.

Language and Technology Stack

After evaluating several options for implementing the intermediate representation and overall system architecture, we have decided to use TypeScript for the entire project stack. This section outlines our evaluation process and rationale.

Language Evaluation

Rust

Advantages:

- Exceptionally robust type system with algebraic data types (enums with associated data)
- Native pattern matching with exhaustive checking and destructuring
- Zero-cost abstractions and excellent performance characteristics
- Memory safety guarantees without garbage collection
- Strong support for functional programming paradigms
- Can compile to WebAssembly for potential browser integration

Disadvantages:

- Steep learning curve, particularly around ownership and borrowing concepts
- Only one team member has prior Rust experience
- Ecosystem instability: frequent breaking changes in frameworks and libraries
- Documentation outside of core language resources is often inconsistent or outdated
- Framework churn would introduce timeline risk in a 12-week project

TypeScript

Advantages:

- Entire team has experience with JavaScript/TypeScript
- Genuinely robust type system with discriminated unions and type narrowing
- Exhaustive type checking at compile time
- Mature, stable ecosystem with well-documented libraries
- Seamless integration between frontend and backend (one language throughout)
- Large community means extensive Stack Overflow/documentation resources
- Fast iteration and development velocity
- End-to-end type safety from database to UI

Disadvantages:

- Not a systems language - lower raw performance than Rust
- Types exist only at compile time, requiring runtime validation for external data
- No native pattern matching syntax
- Less elegant syntax for complex type transformations compared to Rust
- Type system, while sophisticated, requires more verbose type definitions for complex cases

C++

While C++ offers performance characteristics similar to Rust, it is a poor fit for our requirements:

- Type safety: C++ lacks algebraic data types and exhaustive pattern matching. Implementing our IR would require error-prone manual type checking with `dynamic_cast` or visitor patterns
- Memory safety: Manual memory management introduces entire categories of bugs (use-after-free, memory leaks, buffer overflows) that are completely avoided in both Rust and TypeScript
- Modern tooling: C++ build systems and dependency management are notoriously complex compared to cargo (Rust) or npm (TypeScript)
- Development velocity: The combination of manual memory management, weak type safety for sum types, and verbose syntax would significantly slow development

C++ represents the worst of both worlds for our use case: the complexity of a systems language without the safety guarantees of Rust, and none of the developer ergonomics of TypeScript.

Bridging the Gap: ts-pattern

One of TypeScript's main limitations compared to Rust is the lack of native pattern matching syntax. However, the `ts-pattern` library provides sophisticated pattern matching capabilities that closely approximate Rust's experience:

Features provided by ts-pattern:
- Exhaustive pattern matching with compile-time checking
- Destructuring in pattern expressions
- Guard clauses with `P.when()`
- Nested pattern matching for complex data structures
- Type narrowing with full TypeScript integration
- Pattern matching on values, not just types

Example comparison:

```
import { match } from 'ts-pattern';

function renderNode(node: IRNode): string {
  return match(node)
    .with({ kind: 'header', level: 1 }, ({ content }) =>
      `<h1>${content}</h1>`)
    .with({ kind: 'header' }, ({ level, content }) =>
      `<h${level}>${content}</h${level}>`)
    .with({ kind: 'bulletList' }, ({ items }) =>
      `<ul>${items.map(renderBullet).join('')}</ul>`)
    .with({ kind: 'numberedList' }, ({ items }) =>
      `<ol>${items.map(renderNumber).join('')}</ol>`)
    .exhaustive(); // Compiler error if cases are missing
}
```

This provides the functional, type-safe transformation logic that makes Rust pleasant to work with, while maintaining TypeScript's accessibility and ecosystem stability.

ts-pattern details:
- Actively maintained community library

- Mature, stable API
- Adds minimal runtime overhead
- Will be central to our IR transformation pipeline

Final Decision: TypeScript
Given our project constraints and goals, we have chosen TypeScript for the following reasons:

Team and Timeline Alignment: With 12 weeks and only one team member experienced in Rust, TypeScript maximizes our development velocity and allows all team members to contribute effectively from day one.

Technical Sophistication: TypeScript's type system, enhanced with `ts-pattern`, provides the type safety and pattern matching capabilities necessary for sophisticated IR transformations. We can demonstrate technical depth through:
- Complex discriminated union types modeling our IR
- Exhaustive pattern matching for bidirectional transformations
- End-to-end type safety across the entire stack
- Property-based testing for IR transformation correctness

Ecosystem Maturity: The stable TypeScript/Node.js ecosystem reduces risk of mid-project breaking changes, ensuring we can focus on delivering features rather than fighting tooling issues.

Performance Adequacy: For a blog engine, TypeScript's performance is more than sufficient. The IR transformations are not computationally intensive enough to warrant systems-language performance characteristics.

This approach prioritizes delivering a complete, well-architected system on schedule while maintaining the technical sophistication expected of a masters-level project.

Secure Admin Area

Post creator
There would be a few parts to this creator:
- Editor
- (live?) Preview
- Publishing
- Save without publishing
- Save and publish
- Delete/archive

Editor + Preview

Minimum Viable Product:
- A textbox with an iframe next to it that contains the processed contents of the textbox, refreshing on a time delay with some processing in between

Mid-viable product:
- Can we somewhere in between these two options consider an accessibility checker/enforcer or the parser by default making accessibility add-ons? Would forcing them to write a summary or opening paragraph from which metadata/blog post summary be pulled

Maximum Feasible Product:

- A fully embedded live preview and editor in the same pane, similar to what obsidian does, in which only the line being edited is shown as plaintext while the rest is properly rendered

Testing and ownership

Within the time constraints we intend to robustly test the product throughout its development, team ownership for testing will be:

- Unit testing: Mei Happs
- API tests: Mei Happs
- Continuous Integration testing: Mei Happs
- User testing: Paula Blackledge